

Déploiement automatisé d'infrastructures (IaC)

1. Objectifs

déploiement automatique des VMs

déploiement automatique des applications

Technologies qu'on va utiliser dans ce projet: **Debian/Linux, hyperv, Windows, Powershell, Docker, Ansible, git/github, terraform, packer, jenkins**

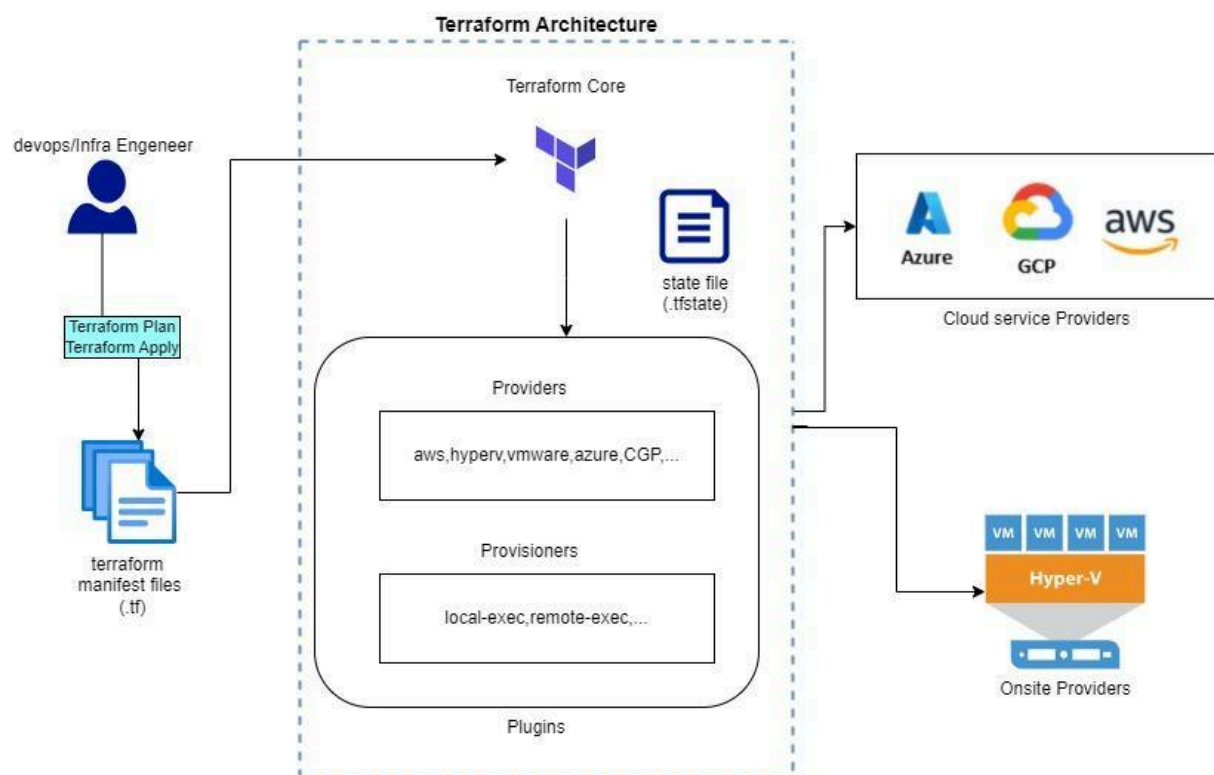
2. IaC et Outils CI/CD

IaC crée et gère les ressources système via des fichiers de configuration au lieu de manipuler les ressources matérielles manuellement ou avec des outils

interactifs. En d'autres termes, IaC signifie automatiser votre infrastructure et traiter les outils automatisés comme du code source.

2.1. Terraform et Ansible

Terraform est un outil d'infrastructure en tant que code (IaC) créé par HashiCorp qui est récemment passé d'une licence open source à une licence BSL. Il permet aux développeurs et aux équipes d'infrastructure de définir et de provisionner une infrastructure à l'aide d'un langage de configuration déclaratif.



L'architecture Terraform se compose principalement des composants suivants :

- **Terraform Core**

Terraform's core (également connu sous le nom de Terraform CLI) est construit sur un binaire compilé statiquement développé à l'aide du langage de programmation Go.

Ce binaire est ce qui génère l'outil de ligne de commande (CLI) connu sous le nom de « Terraform », qui sert d'interface principale pour les utilisateurs de Terraform. Il est open source et est accessible sur Terraform GitHub repository.

- **Providers**

Les Providers Terraform sont des modules qui permettent à Terraform de communiquer avec une gamme diversifiée de services et de ressources, y compris, mais sans s'y limiter, les fournisseurs de cloud, les bases de données et les services DNS.

Chaque Provider est chargé de définir les ressources que Terraform peut gérer au sein d'un service particulier et de traduire les configurations Terraform en appels API spécifiques à ce service.

Exemple de providers :

- Les principaux fournisseurs de cloud comme AWS, Azure et Google Cloud,...
- docker
- vsphere
- Kubernetes
- Hyperv

- **State file**

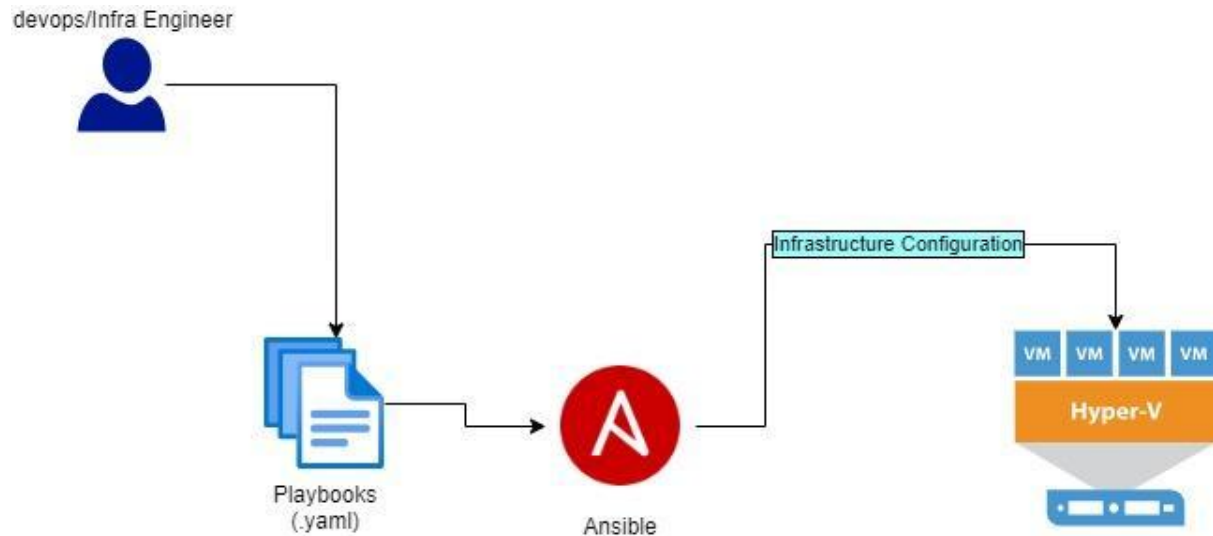
State file est un aspect important du workflow de Terraform. L'état de l'infrastructure est stocké dans un fichier d'état, généralement stocké localement ou de manière centralisée via un backend distant. Il est important de gérer cet état avec soin pour assurer la cohérence de l'infrastructure et éviter les conflits.

Par défaut cet état est stocké dans un fichier local nommé terraform.tfstate, mais il peut également être stocké à distance, mais pas dans un repository git, car il contient des données confidentielles.

Terraform utilise cet état pour créer des plans et apporter des modifications à l'infrastructure. Avant toute opération, Terraform effectue un rafraîchissement pour mettre à jour l'état avec celle de l'infrastructure réelle.

Ansible est un outil open-source d'automatisation des tâches IT. Il permet d'automatiser trois types de tâches : la gestion de configuration, le déploiement d'applications et l'orchestration de tâches. Sa simplicité et sa facilité de prise en main ont fait d'Ansible un outil incontournable dans le monde de l'Infrastructure as Code (IaC). En effet, sa syntaxe en YAML est

simple à appréhender et permet de décrire les tâches à accomplir de manière explicite.



L'intégration de Terraform et Ansible peut prendre différentes formes, mais le scénario commun est l'utilisation de Terraform pour provisionner les ressources, puis le passage de ces ressources à Ansible pour la configuration et l'administration.

En résumé, en combinant les forces de Terraform et Ansible, vous pouvez automatiser de bout en bout le cycle de vie de vos infrastructures, de la création des ressources à la configuration et à l'administration des systèmes. Cela permet non seulement un déploiement plus rapide et plus fiable, mais aussi une gestion plus facile et des économies de coûts significatives.

2.2. Documentation utiles

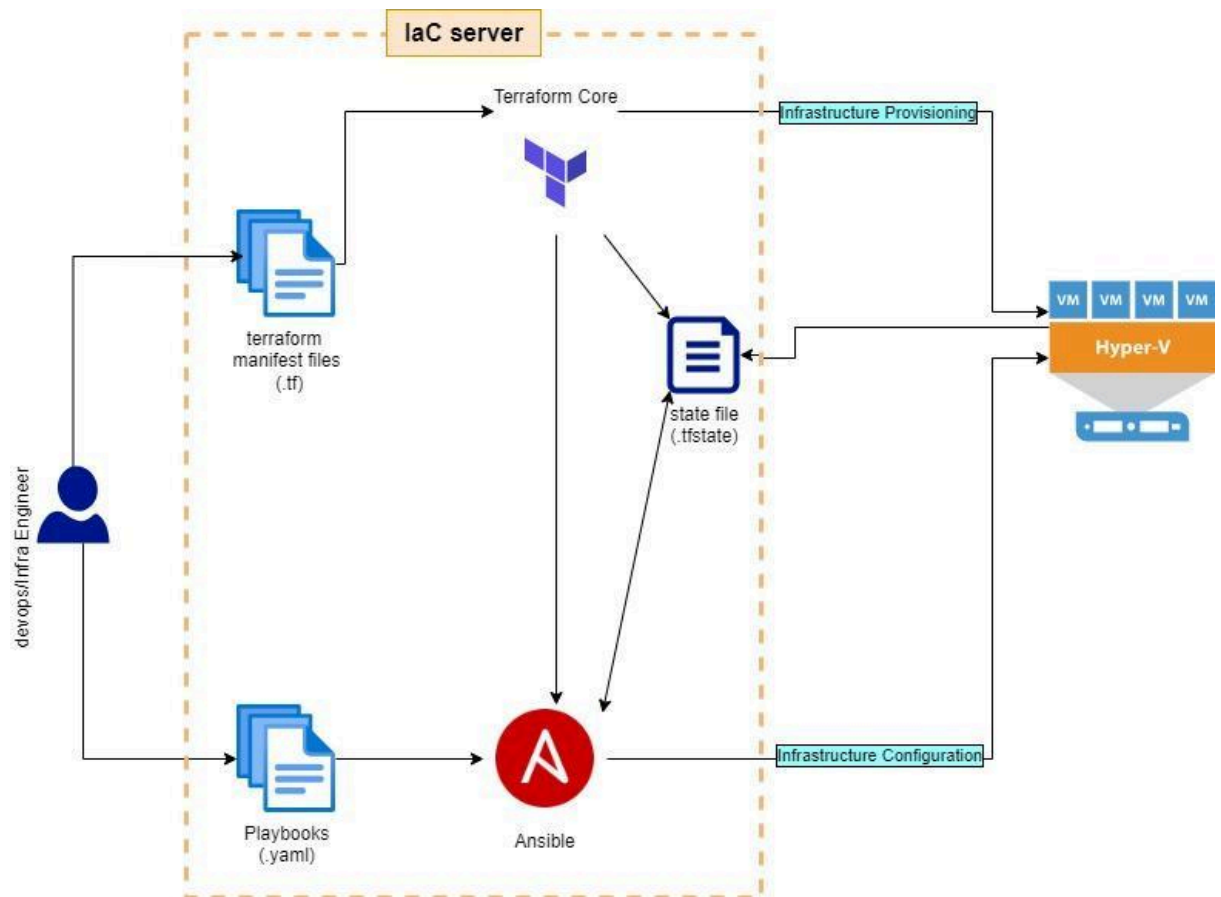
<https://spacelift.io/blog/terraform-architecture>

<https://blog.wescale.fr/ansible-packer-terraform-un-trio-gagnant>

3. Phase 1 : Création et configuration VM sur hyperv en utilisant terraform et ansible

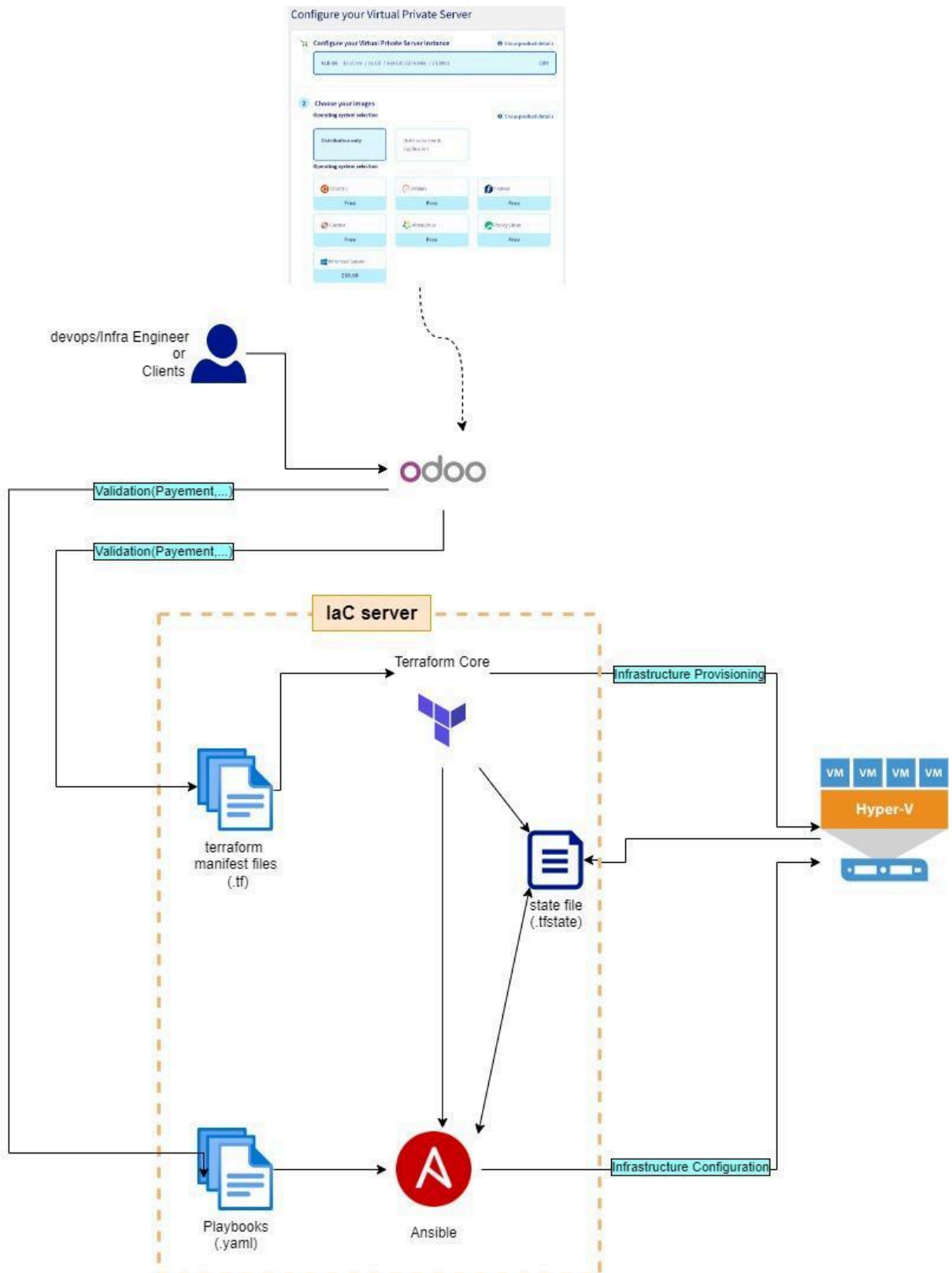
- création des templates VM : windows (server 2019, server 2022), linux (debian, ubuntu, centos), docker VM avec odoo (ubuntu)
- installation du IaC server avec terraform et ansible
- choix hyperv de test
- configuration terraform
- configuration ansible
- passage par git (github ensuite gitlab local) pour les manifest files et playbooks

- tests et validation
- création template VM utilisant packer
- documentation



4. Phase 2: interface graphique odoo avec backend terraform/ansible

- Interface IaC server avec odoo: création POS sur odoo
- Une validation sur odoo d'une commande va exécuter le provisioning ou/et la configuration sur le serveur IaC



-----NOTES Manitra-----

install packer

<https://developer.hashicorp.com/packer/install>